# ROBOTS MEET ARTS

## EDUCATIONAL ROBOTICS AND CODING MEET ARTS AND HUMANITIES IN INCLUSIVE LEARNING ENVIRONMENTS

# MODULE 6

## KNOWLEDGE PUT INTO PRACTICE & PRESENTATIONS

COORDINATOR & PARTNERS

Co-funded by the European Union

Co funded by the European Union (Project number: 2023-1-TR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

# Theory and key concepts

## Introduction

As we arrive at the final stage of the Robots Meet Arts training, the focus shifts from exploring theory and methods to putting knowledge into action. This module supports you in designing and presenting your own educational robotics and coding lesson plan—grounded in everything you've learned so far.

To do this effectively, it's essential to consolidate key concepts, reflect on what makes a robotics activity meaningful, and equip yourself with the practical tools needed to plan, facilitate, and explain robotics-enhanced lessons in your own teaching context. This section provides the pedagogical foundation for your upcoming group work, offering you a toolkit of teaching tips and a glossary of essential terminology to guide your thinking.

Co-funded by the European Union

Co-funded by the European Union (Project number: 2023-1-IR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

# PART 1 -Tips for Lesson Design with Educational Robotics and Coding

Designing a lesson that meaningfully incorporates robotics and coding doesn't require high-level programming skills—it requires intentional planning, clear learning goals, and a student-centered mindset. Below are expanded guidelines to support teachers as they move from theory to practice:

## 1. Start with Clear Learning Objectives

Before selecting a robot or coding tool, define what you want students to learn. Is your lesson focused on narrative development? Environmental awareness? Geometry? Teamwork?

✅ Example: In a literature class, the objective might be "Students will practice their oral and programming skills by retelling the plot of a story using robotic movement and voice narration."

Tip: Align objectives with your national curriculum or cross-curricular themes to ensure the activity has pedagogical value beyond novelty.

## 2. Anchor the Activity in a Story, Theme, or Real-World Problem

Robotics becomes more meaningful when it's contextualized. Framing the task as part of a story or authentic challenge increases student motivation and relevance.

✅ Example: "Your robot is a journalist traveling across a country to collect cultural facts."
✅ Example: "You will design a robot to help deliver medicine in a disaster zone simulation."

Tip: Story-driven or problem-based tasks naturally support cross-disciplinary learning and creativity.

Co-funded by the European Union

Co-funded by the European Union (Project number: 2023-1-IR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

## 3. Choose the Right Tools for the Age and Objectives

Match the complexity of the technology with the cognitive level and experience of your students. Young learners benefit from tactile, button-based robots (like Bee-Bot or Tale-Bot), while older students can handle more open-ended platforms like LEGO SPIKE or Scratch.

✅ Early years: mTiny, Bee-Bot, Cubetto
✅ Primary school: Tale-Bot, LEGO SPIKE Essential, Scratch Jr.
✅ Upper primary/secondary: Scratch, LEGO SPIKE Prime, Micro:bit

Tip: If possible, allow students to explore the tools informally before the lesson begins to reduce tech anxiety.

## 4. Use Scaffolded Steps with Optional Extensions

Structure the lesson so that all students can engage with the basic task, but also offer "challenge extensions" for those who are ready.

✅ Step 1: Program your robot to move from point A to B
✅ Step 2: Add a voice line or sound at key locations
✅ Step 3: Include a loop or condition ("If I reach the bridge, turn right")
✅ Extension: Add sensors, a score-keeping system, or a second character

Tip: Scaffolded tasks promote inclusion and allow mixed-ability groups to work collaboratively.

## 5. Incorporate Student Voice and Choice

Giving students choices—in story paths, design decisions, or presentation formats—boosts ownership, motivation, and creativity.

✅ Allow students to choose the theme (e.g., farm, space, city)
✅ Let them decide how their robot expresses itself (sound, movement, costumes)

Tip: Use open-ended project briefs with clear boundaries. Example: "Your robot must meet three characters and ask them one question each

Co-funded by the European Union

Co-funded by the European Union (Project number: 2023-1-IR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

## 6. Anticipate Diverse Needs

To create an inclusive lesson, differentiate the process, product, or content based on your students' learning profiles. Use visuals, physical prompts, peer support, and simplified instructions where needed.

✅ Pair non-readers with verbal teammates.
✅ Provide printed coding cards with symbols.
✅ Use voice-based robots for students with reading difficulties.

Tip: Design with Universal Design for Learning (UDL) principles in mind: offer multiple means of engagement, representation, and expression.

## 7. Integrate Reflection Moments

Build in short, meaningful reflection points where students pause to think about their progress, challenges, and learning strategies.

✅ Use prompts like:

- "What did we change and why?"
- "What did we learn from our mistake?"
- "What would we do differently next time?"

Tip: Use simple exit tickets, group debriefs, or drawings to capture reflections—especially for younger students.

Designing lessons that integrate educational robotics and coding **is not about mastering technology—it's about creating engaging, inclusive, and meaningful learning experiences where students are empowered to explore, create, and express themselves.** By starting with clear goals, choosing the right tools, scaffolding learning, and inviting student voice, teachers can transform robotics into a flexible teaching ally across subjects and age groups. When students practice coding with purpose, build with creativity, and share their learning with confidence, they don't just learn robotics—they learn to think critically, collaborate effectively, and take ownership of their learning journey.

Co-funded by the European Union

Co-funded by the European Union (Project number: 2023-1-IR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

# PART 2 -Key Terms for Teaching with Robots

## Coding & Programming Concepts

- Algorithm: A step-by-step set of instructions to complete a task.
- Command: A single instruction given to the robot (e.g., "move forward").
- Sequence: The specific order in which commands are given and executed.
- Loop: A programming structure that repeats a set of instructions.
- Condition / Conditional: An instruction that only runs if a certain rule is met (e.g., "If button is pressed, then move").
- Debugging: Finding and fixing errors or problems in the code.
- Input: Information or signals sent to the robot (e.g., sensor data).
- Output: What the robot does in response to code (e.g., movement, sound).
- Variable: A named value that can change while the program is running.
- Sensor: A tool the robot uses to detect things in its environment (e.g., distance sensor, color sensor, sound sensor).

## Robotics-Specific Vocabulary

- Motor: The part that drives movement (wheels, arms, etc.).
- Servo: A motor that rotates to a specific position, useful for controlled motion.
- Controller: The “brain” of the robot, often a microcontroller like a LEGO Hub or Micro:bit.
- Autonomous: A robot that follows its programmed instructions without needing direct human control.
- Remote control: A method for manually controlling a robot via buttons or apps.
- Simulation: Testing robot behavior in a virtual or imagined environment before using it in real life.

## Pedagogical & Learning Terms

- Project-based learning (PBL): An approach where students learn through designing and completing projects.
- Collaboration: Working in groups to solve problems and complete tasks.
- Iteration: Making repeated improvements based on testing and feedback.
- Scaffolding: Supporting students with guidance that is gradually reduced as they become more confident.
- Reflection: Thinking back on what was done, what was learned, and what could be improved.
- Cross-curricular: Integrating learning across multiple subjects (e.g., combining robotics with storytelling or geography).

Co-funded by the European Union

Co-funded by the European Union (Project number: 2023-1-IR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.